PES UNIVERSITY

Department of Computer Science & Engineering

Database Management System

UE23CS351A

Mini Project – Report

Name of the Student	**P Vishnupranav Reddy**
SRN	**PES1UG23CS415**
Section	**G**
Name of the Student	**Pavan K Aithal**
SRN	**PES1UG23CS420**
Section	**G**

Indian Energy Monitoring System

The Indian Energy Monitoring System is a comprehensive web-based platform designed to track, analyse, and visualize India's energy generation landscape in real-time. The system aggregates live data from ‘meritindia.in’ and ‘cea.nic.in’, capturing daily generation statistics from renewable and conventional power plants across multiple states and regions. Built with a React-based frontend and Flask backend architecture, the platform provides users with interactive dashboards, analytical insights, and automated alert mechanisms for monitoring energy production patterns. The system maintains a structured database of power plants, companies, and regional metrics, enabling comparative analysis of energy mix, generation trends, and capacity utilization. By processing daily XLS and PDF reports through automated web scraping, the platform transforms raw energy data into actionable intelligence. The application serves as a centralized monitoring tool for understanding India's dynamic energy ecosystem and facilitating data-driven insights into the nation's power generation infrastructure.

User Requirement Specifications:

Functional Requirements:

1. **Data Acquisition and Preprocessing:**

   • **Real-time Data Scraping:** Automated web scraping from meritindia.in to fetch daily power generation reports
   • **Multi-format Support:** Process XLS (plant generation) and PDF (renewable energy) reports
   • **Scheduled Updates:** Daily synchronization of power plant generation data from GitHub repository
   • **Data Parsing Pipeline:** Sequential execution of integrated_web_scrapping.py, parseall1.py, parseall2.py, and parseall3.py

2. **Dashboard and Visualisation:**

   Key Performance Indicators (KPIs):

   The main dashboard focuses on several critical KPIs to ensure immediate insight into the current state of operations. These metrics include the Total number of power plants under management and the overall system performance, specifically Today's generation (MU - Million Units) and Today's demand (MU). Operational health is monitored through the Average efficiency percentage and the Total operational capacity (MW - Megawatts). Furthermore, the dashboard highlights potential immediate issues by tracking the Plants under outage count and Critical coal stock alerts. To facilitate historical review and flexible analysis, the dashboard incorporates a crucial Date-based filtering capability.

   Interactive Charts:

   A suite of interactive visualizations supports the raw KPI data, allowing users to delve deeper into performance. The Energy mix pie chart clearly illustrates the composition of the power portfolio by breaking down the generation sources into renewable versus non-renewable categories. Performance can be analysed geographically using Regional performance bar charts and a Map visualization for geographical distribution of the plants. For strategic and historical analysis, the dashboard features Trend line charts for historical analysis of key metrics. Finally, a dedicated visualization provides a Top performing plants comparison to benchmark and identify best-in-class operations.

3. **Plant Management:**

   Plant Listing:

   The Plant Listing serves as the primary index for all power generating facilities. It uses a Paginated view (20 plants per page) to ensure quick loading and easy navigation across the entire fleet. To facilitate rapid location of specific facilities, the list includes Search functionality by plant name/ID. Advanced data segmentation is supported via several filters, allowing users to Filter by state, sector (State/Private/Central), and energy type. Each entry in the listing is designed to Display basic plant information for immediate identification and context.

   Detailed Plant View:

   Selecting a plant from the listing opens the Detailed Plant View, which aggregates all relevant operational and performance data. This view begins with Comprehensive plant information (name, location, sector, energy type). It provides an in-depth look at performance through the Production history with efficiency metrics and details the physical capability with Operational capacity and capability. For thermal plants, essential resource monitoring is provided via Coal stock levels. The operational status is shown at a granular level with Operational status by unit and critical Outage information with expected sync dates. All data within this view is supported by a Date-based historical data viewing feature, allowing users to analyse performance trends over time.

4. **Analytics and Insights:**

   Regional Performance Analysis:

This section offers a geographical perspective on the power system's operation. It starts by quantifying the infrastructure with the Region-wise total plants count. Core performance is measured by breaking down Generated energy (MU) and Demand (MU) by region. The financial and operational balance is determined through Surplus/deficit calculation. Further supply health is measured by Supply percentage metrics and a clear Energy status classification (e.g., meeting demand, low surplus, high deficit) for quick assessment of regional stability.

Efficiency Comparison:

The Efficiency Comparison provides a competitive and developmental perspective on plant operations. It ranks all facilities using a Plant-wise average efficiency ranking. A focused list highlights the high achievers with the Top 20 performing plants. To identify performance trends and the impact of operational changes, the feature includes a view of Efficiency percentage over time for individual plants.

Renewable Energy Mix:

Dedicated analysis of sustainable power generation is provided in the Renewable Energy Mix section. It breaks down the system's energy sources on a granular level with a State-wise renewable vs non-renewable breakdown. This data is then used to calculate the crucial Renewable percentage calculation to track sustainability goals. The raw contribution of these sources is quantified in Production values in MU. For recent trend analysis, all metrics in this section are based on Last 30-day aggregated data.

5. **Alert System:**

Real-time Monitoring Alerts:

- Low Efficiency Alerts: Plants with <60% average efficiency (3-day window)
- Critical Coal Stock: Thermal plants with <7 days coal stock (WARNING: 4-7 days, CRITICAL: <4 days)
- Plant Outage Alerts: Units under outage status (CRITICAL: ≥500 MW, WARNING: <500 MW)

Alert Dashboard:

The Alert Dashboard provides a focused, real-time view of system issues. The primary metric is the Total alerts count, giving a quick measure of overall system risk. Alerts are then prioritized through Critical vs warning categorization to guide immediate response efforts. Each notification is presented with Detailed alert information which includes the associated plant name, state, and the values that triggered the alert. Finally, a dedicated Coal stock table provides immediate context for thermal plant issues by displaying the current coal levels alongside the plant’s operational capacity, allowing operators to quickly assess the severity of fuel shortages relative to production capability.

6. **Regional/State Management;**

This feature set is designed to offer a clear, structured view of how power generation and consumption are balanced across different geographical areas. Key metrics are reported at the state level:

- **Plant count per state:** The total number of operational power plants located within each state, providing context for the infrastructure base.
- **Generated energy:** The total power produced (typically measured in MU - Million Units) by all plants within a specific state.
- **Demand:** The total power consumed (MU) within the specific state.
- **Energy status (surplus/deficit):** A calculated metric showing the difference between the generated energy and demand (Generation - Demand), clearly indicating whether the state is an energy supplier (surplus) or a net importer (deficit).
- **Regional grouping:** States are aggregated into macro-regions—Northern, Southern, Eastern, Western, and North-Eastern—to provide a high-level overview of inter-regional performance and balance.
- **Date-based filtering:** A necessary tool that allows users to analyse these state and regional metrics over specific timeframes (e.g., daily, weekly, or monthly comparisons).

7. **Database Administration:**

   • **Daily Report Generation:** Generate comprehensive daily energy reports by date with state-wise aggregated metrics
   • **Underperforming Plant Identification:** Configurable efficiency threshold and time window
   • **Data Pipeline Trigger:** Manual trigger for complete ETL process with error handling.

Tech Stack:

Backend:

• Framework: Flask 3.0.0

• Database: MySQL with SQL Alchemy ORM

• API Architecture: RESTful APIs with Blueprint modularization

Frontend:

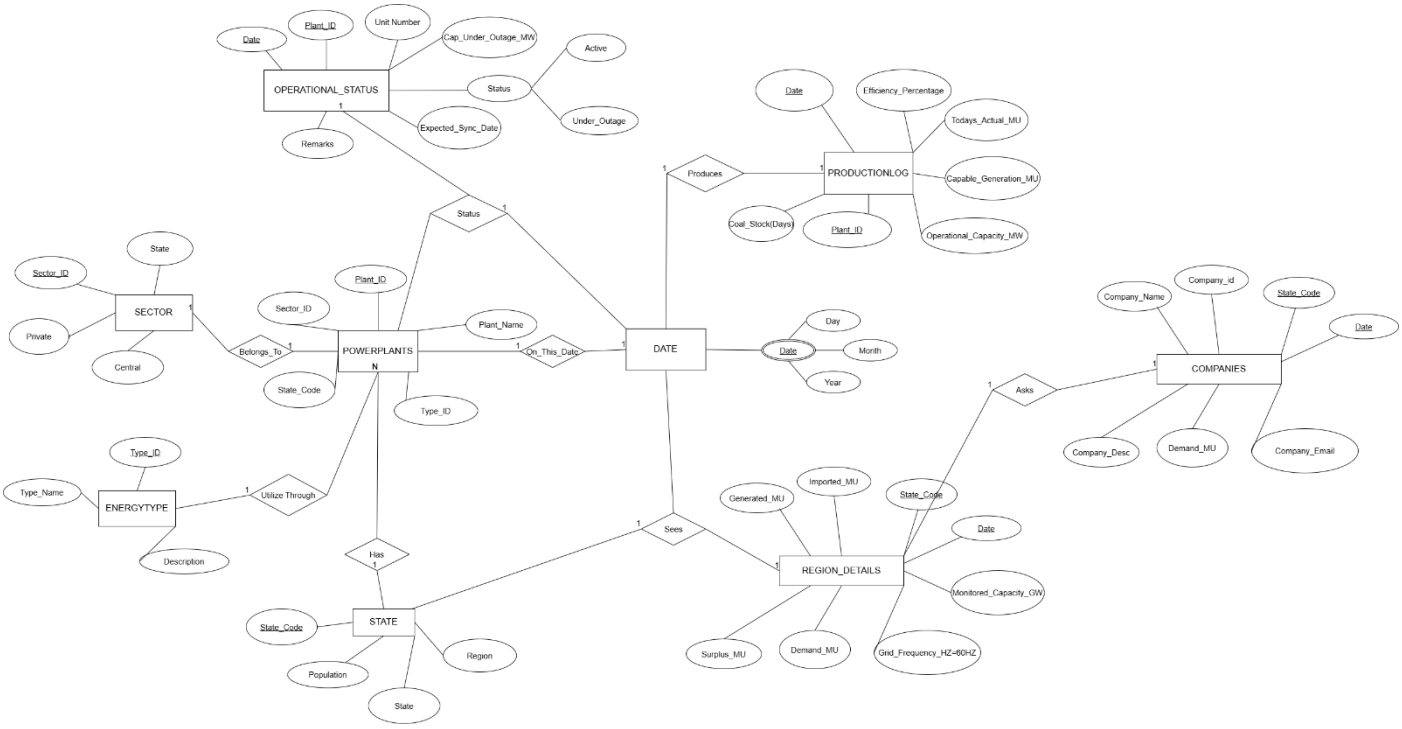
• Framework: React 18.3.1

• Routing: React Router DOM 7.9.5

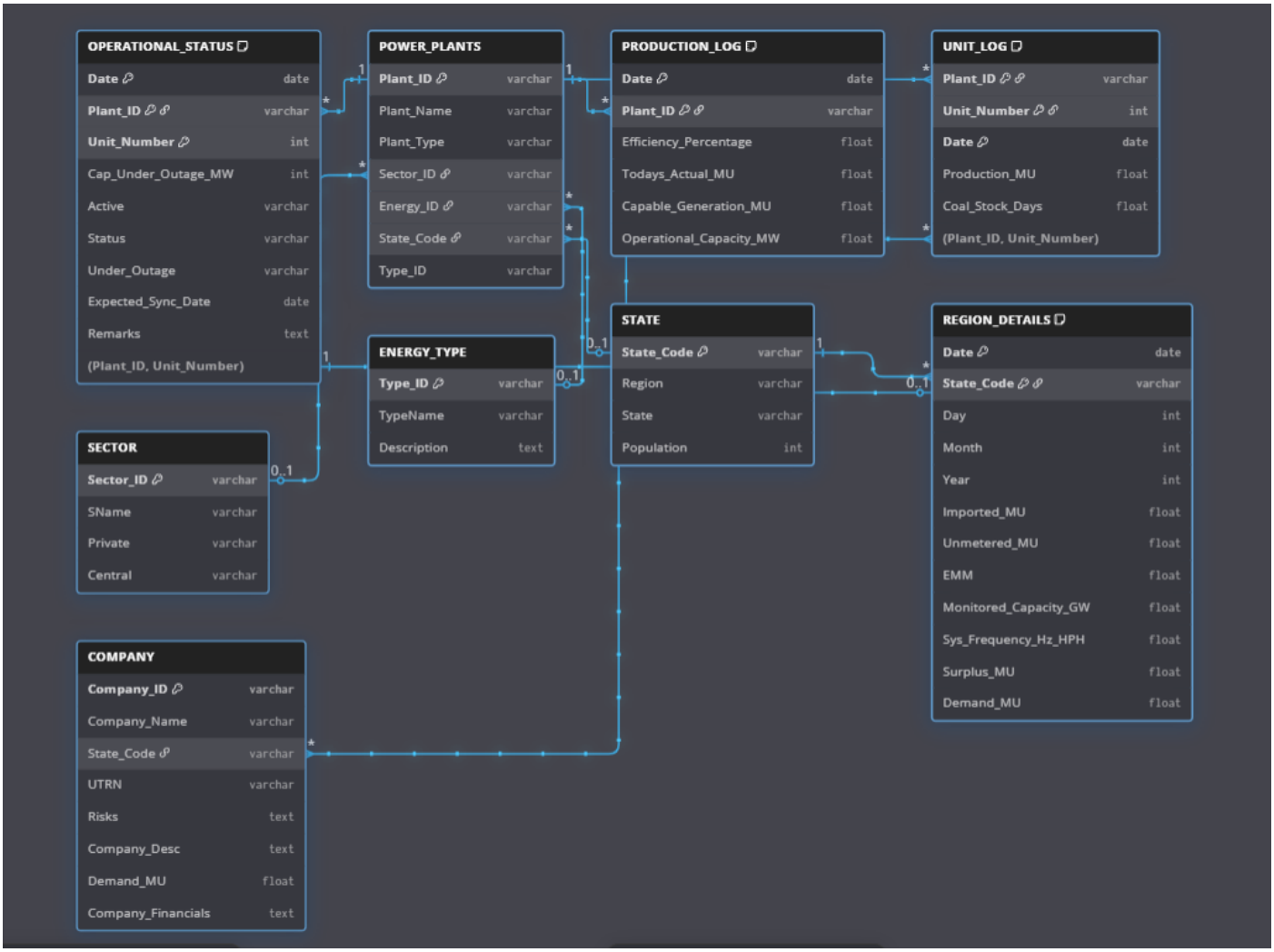
• Charts: Recharts 3.3.0

• HTTP Client: Axios 1.13.2

E R Diagram:



Relational Schema:



DDL Commands:

```
CREATE DATABASE IF NOT EXISTS IndianEnergyDB;
USE IndianEnergyDB;

-- STATE Entity
CREATE TABLE STATE (
    State_Code VARCHAR(10) PRIMARY KEY,
    State_Name VARCHAR(100),
    Region VARCHAR(100),
    Population BIGINT
);


-- SECTOR Entity
CREATE TABLE SECTOR (
    Sector_ID VARCHAR(10) PRIMARY KEY,
    Sector_Name VARCHAR(100) NOT NULL -- e.g., 'State', 'Private', 'Central'
);

-- ENERGYTYPE Entity
CREATE TABLE ENERGYTYPE (
    Type_ID VARCHAR(10) PRIMARY KEY,
    Type_Name VARCHAR(100),
    Description TEXT
);

-- DATE_DIM Entity
CREATE TABLE DATE_DIM (
    `Date` DATE PRIMARY KEY,
    Day INT,
    Month INT,
    Year INT
);

CREATE TABLE POWERPLANTS (
    Plant_ID VARCHAR(20) PRIMARY KEY,
    Plant_Name VARCHAR(255) NOT NULL,
    State_Code VARCHAR(10),
    Sector_ID VARCHAR(10),
    Type_ID VARCHAR(10),

    FOREIGN KEY (State_Code) REFERENCES STATE(State_Code),
    FOREIGN KEY (Sector_ID) REFERENCES SECTOR(Sector_ID),
    FOREIGN KEY (Type_ID) REFERENCES ENERGYTYPE(Type_ID)
);
```

```sql
-- OPERATIONAL_STATUS Entity (Weak entity, depends on POWERPLANTS and DATE_DIM)
CREATE TABLE OPERATIONAL_STATUS (
    Plant_ID VARCHAR(20),
    Unit_Number VARCHAR(20),
    Status_Date DATE,
    Cap_Under_Outage_MW DECIMAL(10, 2),
    Status ENUM('Active', 'Under Outage', 'Not Commisioned'),
    Outage_Date DATE,
    Expected_Sync_Date DATE,
    Remarks TEXT,

    PRIMARY KEY (Plant_ID, Unit_Number, Status_Date),

    FOREIGN KEY (Plant_ID) REFERENCES POWERPLANTS(Plant_ID),
    FOREIGN KEY (Status_Date) REFERENCES DATE_DIM(`Date`)
);

-- REGION_DETAILS Entity (Fact table linking STATE and DATE_DIM)
CREATE TABLE REGION_DETAILS (
    State_Code VARCHAR(10),
    Report_Date DATE,
    Generated_MU DECIMAL(12, 2),
    Imported_MU DECIMAL(12, 2),
    Surplus_MU DECIMAL(12, 2),
    Demand_MU DECIMAL(12, 2),
    Monitored_Capacity_MW DECIMAL(10, 2),
    Grid_Frequency_HZ DECIMAL(5, 2),

    PRIMARY KEY (State_Code, Report_Date),
    FOREIGN KEY (State_Code) REFERENCES STATE(State_Code),
    FOREIGN KEY (Report_Date) REFERENCES DATE_DIM(`Date`)
);

-- PRODUCTIONLOG Entity (Fact table linking POWERPLANTS and DATE_DIM)
CREATE TABLE PRODUCTIONLOG (
    Plant_ID VARCHAR(20),
    Log_Date DATE,
    Efficiency_Percentage DECIMAL(6, 2),
    Todays_Actual_MU DECIMAL(12, 2),
    Capable_Generation_MU DECIMAL(12, 2),
    Operational_Capacity_MW DECIMAL(10, 2),
    Coal_Stock_Days DECIMAL(10, 2),

    PRIMARY KEY (Plant_ID, Log_Date),
    FOREIGN KEY (Plant_ID) REFERENCES POWERPLANTS(Plant_ID),
    FOREIGN KEY (Log_Date) REFERENCES DATE_DIM(`Date`)
);
```

Triggers/Functions/Procedures:

```
DELIMITER $$

CREATE TRIGGER trg_region_demand_before_insert
BEFORE INSERT ON REGION_DETAILS
FOR EACH ROW
BEGIN
    -- Check if the incoming Demand_MU value is not NULL
    -- This value is assumed to be in MW
    IF NEW.Demand_MU IS NOT NULL THEN

        -- Step 1: Convert the value from MW to Million Units (MU)
        -- (MW * 24 hours) / 1000 MWh/MU
        SET NEW.Demand_MU = NEW.Demand_MU * 0.024;

    END IF;
END$$

DELIMITER ;

DELIMITER $$

CREATE PROCEDURE sp_UpdateRegionGenerationFromProduction()
BEGIN
    -- This query joins REGION_DETAILS with an aggregated subquery
    -- that calculates the total generation (sum of Todays_Actual_MU)
    -- for each state and date from the PRODUCTIONLOG.

    UPDATE REGION_DETAILS AS rd
    JOIN (
        -- Subquery: Calculate the total actual generation for each state and date
        SELECT
            p.State_Code,
            pl.Log_Date,
            SUM(pl.Todays_Actual_MU) AS Total_Actual_MU
        FROM
            PRODUCTIONLOG AS pl
        JOIN
            POWERPLANTS AS p ON pl.Plant_ID = p.Plant_ID
        WHERE
            p.State_Code IS NOT NULL
        GROUP BY
            p.State_Code, pl.Log_Date
    ) AS daily_totals
    ON
        rd.State_Code = daily_totals.State_Code
        AND rd.Report_Date = daily_totals.Log_Date

    -- Set the Generated_MU column to the calculated total
    SET
        rd.Generated_MU = daily_totals.Total_Actual_MU;
END$$

DELIMITER ;
```

```
DELIMITER $$

CREATE PROCEDURE sp_UpdateRegionSurplusAndImports()
BEGIN
    UPDATE REGION_DETAILS
    SET
        Surplus_MU = CASE
                         WHEN Generated_MU > Demand_MU THEN (Generated_MU - Demand_MU)
                         ELSE 0
                     END,

        Imported_MU = CASE
                          WHEN Demand_MU > Generated_MU THEN (Demand_MU - Generated_MU)
                          ELSE 0
                      END
    WHERE
        Generated_MU IS NOT NULL
        AND Demand_MU IS NOT NULL;

END$$

DELIMITER ;

DELIMITER $$

CREATE PROCEDURE sp_CalculatePlantEfficiency()
BEGIN
    UPDATE PRODUCTIONLOG
    SET
        -- Calculating efficiency
        Efficiency_Percentage = (Todays_Actual_MU / Capable_Generation_MU) * 100
    WHERE
        -- Preventing division by zero errors
        Capable_Generation_MU IS NOT NULL
        AND Capable_Generation_MU > 0
        AND Todays_Actual_MU IS NOT NULL;
END$$

DELIMITER ;
```

```
DELIMITER $$

CREATE PROCEDURE sp_InsertAllMissingActiveStatuses()
BEGIN
    -- Insert 'Active' status records for plant/date combinations
    -- that have NO entries at all in OPERATIONAL_STATUS.

    INSERT INTO OPERATIONAL_STATUS (
        Plant_ID,
        Unit_Number,
        Status_Date,
        Status,
        Cap_Under_Outage_MW,
        Expected_Sync_Date,
        Remarks,
        Outage_Date
    )
    -- Select all (Plant, Date) combinations...
    SELECT
        p.Plant_ID,
        'Main' AS Unit_Number,
        d.`Date` AS Status_Date,
        'Active' AS Status,
        0.00 AS Cap_Under_Outage_MW,
        NULL AS Expected_Sync_Date,
        NULL AS Remarks,
        NULL AS Outage_Date
    FROM
        POWERPLANTS p
    CROSS JOIN
        DATE_DIM d
    WHERE
        -- ...for which NOT A SINGLE record exists in OPERATIONAL_STATUS
        -- for that Plant_ID and Date (regardless of Unit_Number)
        NOT EXISTS (
            SELECT 1
            FROM OPERATIONAL_STATUS os
            WHERE os.Plant_ID = p.Plant_ID
              AND os.Status_Date = d.`Date`
        );
END$$

DELIMITER ;
```

```
DELIMITER $$
CREATE PROCEDURE sp_GenerateDailyEnergyReport(IN report_date DATE)
BEGIN
-- Safe aggregation: read totals directly from REGION_DETAILS (one row per state/date)
-- This avoids row-multiplication caused by joining POWERPLANTS/PRODUCTIONLOG
-- before aggregation.
SELECT
rd.State_Code,
rd.Report_Date,
COALESCE(rd.Generated_MU, 0) AS total_generated_mu,
COALESCE(rd.Demand_MU, 0) AS total_demand_mu,
COALESCE(rd.Surplus_MU, 0) AS total_surplus_mu,
COALESCE(rd.Imported_MU, 0) AS total_imported_mu
FROM REGION_DETAILS rd
WHERE rd.Report_Date = report_date
ORDER BY rd.State_Code;
END$$
DELIMITER ;

-- 2) Stored procedure: sp_IdentifyUnderperformingPlants
DELIMITER $$
CREATE PROCEDURE sp_IdentifyUnderperformingPlants(IN threshold_efficiency DECIMAL(5,2), IN days_to_check INT)
BEGIN
    SELECT
        p.Plant_ID,
        p.Plant_Name,
        s.State_Name,
        ROUND(AVG(pl.Efficiency_Percentage), 2) AS avg_efficiency
    FROM POWERPLANTS p
    LEFT JOIN PRODUCTIONLOG pl ON p.Plant_ID = pl.Plant_ID
    LEFT JOIN STATE s ON p.State_Code = s.State_Code
    WHERE pl.Log_Date >= DATE_SUB(CURDATE(), INTERVAL days_to_check DAY)
    GROUP BY p.Plant_ID, p.Plant_Name, s.State_Name
    HAVING AVG(pl.Efficiency_Percentage) < threshold_efficiency
    ORDER BY avg_efficiency ASC;
END$$
DELIMITER ;
```

```
DELIMITER $$
CREATE PROCEDURE sp_CalculateRegionalMetrics(IN start_date DATE, IN end_date DATE)
BEGIN
    -- Aggregate production data by state first
    WITH StateProduction AS (
        SELECT
            p.State_Code,
            COUNT(DISTINCT p.Plant_ID) AS Total_Plants,
            COALESCE(SUM(pl.Todays_Actual_MU), 0) AS Total_Generated_MU
        FROM POWERPLANTS p
        LEFT JOIN PRODUCTIONLOG pl ON p.Plant_ID = pl.Plant_ID
            AND pl.Log_Date BETWEEN start_date AND end_date
        GROUP BY p.State_Code
    ),
    -- Aggregate region details by state separately
    StateDemand AS (
        SELECT
            State_Code,
            COALESCE(SUM(Demand_MU), 0) AS Total_Demand_MU
        FROM REGION_DETAILS
        WHERE Report_Date BETWEEN start_date AND end_date
        GROUP BY State_Code
    )
    -- Now joining the pre-aggregated results
    SELECT
        s.Region,
        COALESCE(SUM(sp.Total_Plants), 0) AS Total_Plants,
        COALESCE(SUM(sp.Total_Generated_MU), 0) AS Total_Generated_MU,
        COALESCE(SUM(sd.Total_Demand_MU), 0) AS Total_Demand_MU,
        CASE
            WHEN SUM(sd.Total_Demand_MU) > 0 THEN
                ROUND((SUM(sp.Total_Generated_MU) / SUM(sd.Total_Demand_MU)) * 100, 2)
            ELSE 0
        END AS Supply_Percentage
    FROM STATE s
    LEFT JOIN StateProduction sp ON s.State_Code = sp.State_Code
    LEFT JOIN StateDemand sd ON s.State_Code = sd.State_Code
    GROUP BY s.Region
    ORDER BY Total_Generated_MU DESC;
END$$
DELIMITER ;
```

```
-- Function 1: Calculate Renewable Percentage
DELIMITER $$
CREATE FUNCTION fn_calculate_renewable_percentage(
    renewable_mu DECIMAL(15,2),
    total_mu DECIMAL(15,2)
)
RETURNS DECIMAL(5,2)
DETERMINISTIC
BEGIN
    RETURN ROUND((renewable_mu / NULLIF(total_mu, 0)) * 100, 2);
END$$
DELIMITER ;

-- Function 2: Classify Energy Category
DELIMITER $$
CREATE FUNCTION fn_energy_category(energy_type VARCHAR(50))
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
    IF energy_type IN ('HYDRO', 'WIND', 'SOLAR', 'BIOMASS') THEN
        RETURN 'Renewable';
    ELSE
        RETURN 'Non-Renewable';
    END IF;
END$$
DELIMITER ;

-- Function 3: Determine Coal Stock Alert Severity
DELIMITER $$
CREATE FUNCTION fn_coal_stock_severity(coal_days INT)
RETURNS VARCHAR(10)
DETERMINISTIC
BEGIN
    IF coal_days < 4 THEN
        RETURN 'CRITICAL';
    ELSEIF coal_days < 7 THEN
        RETURN 'WARNING';
    ELSE
        RETURN 'NORMAL';
    END IF;
END$$
DELIMITER ;
```

Nested Queries/Corelated Queries/Aggregate Queries/Joins:

```
Get energy type distribution for last 30 days

SELECT COUNT(DISTINCT Plant_ID) FROM POWERPLANTS;
SELECT Log_Date FROM PRODUCTIONLOG GROUP BY Log_Date HAVING COUNT(DISTINCT Plant_ID) = :total ORDER BY Log_Date DESC LIMIT 1;
SELECT MAX(d.`Date`) FROM DATE_DIM d WHERE EXISTS (SELECT 1 FROM PRODUCTIONLOG pl WHERE pl.Log_Date = d.`Date;

SELECT
COALESCE(et.Type_Name, 'Unknown') as type_name,
COUNT(DISTINCT p.Plant_ID) as plant_count,
COALESCE(SUM(pl.Todays_Actual_MU), 0) as total_generation_mu,
COALESCE(ROUND(AVG(pl.Efficiency_Percentage), 2), 0) as avg_efficiency
FROM ENERGYTYPE et
LEFT JOIN POWERPLANTS p ON et.Type_ID = p.Type_ID
LEFT JOIN PRODUCTIONLOG pl ON p.Plant_ID = pl.Plant_ID
AND pl.Log_Date = :log_date
WHERE et.Type_ID IS NOT NULL
GROUP BY et.Type_Name
HAVING SUM(pl.Todays_Actual_MU) > 0
ORDER BY total_generation_mu DESC;

Get top performing plants

SELECT
    p.Plant_ID,
    p.Plant_Name,
    s.State_Name,
    et.Type_Name,
    COALESCE(ROUND(pl.Efficiency_Percentage, 2), 0) AS efficiency,
    COALESCE(ROUND(pl.Todays_Actual_MU, 2), 0) AS todays_generation_mu
FROM POWERPLANTS p
LEFT JOIN STATE s
    ON p.State_Code = s.State_Code
LEFT JOIN ENERGYTYPE et
    ON p.Type_ID = et.Type_ID
LEFT JOIN PRODUCTIONLOG pl
    ON p.Plant_ID = pl.Plant_ID
   AND pl.Log_Date = :log_date
ORDER BY efficiency DESC
LIMIT 10;
```

```
Get all regions with energy statistics

SELECT
    rd.State_Code,
    s.State_Name,
    s.Region,
    (SELECT COUNT(DISTINCT Plant_ID)
     FROM POWERPLANTS
     WHERE State_Code = rd.State_Code) AS Plant_Count,

    COALESCE(rd.Generated_MU, 0) AS Generated_MU,
    COALESCE(rd.Demand_MU, 0) AS Demand_MU,

    CASE
        WHEN COALESCE(rd.Generated_MU, 0) >= COALESCE(rd.Demand_MU, 0)
            THEN 'Surplus'
        ELSE 'Deficit'
    END AS Energy_Status

FROM REGION_DETAILS rd
INNER JOIN STATE s
    ON rd.State_Code = s.State_Code

WHERE rd.Report_Date = :selected_date
ORDER BY s.Region, s.State_Name;

Get energy mix distribution for a specific state

SELECT
    et.Type_Name,
    COALESCE(SUM(pl.Todays_Actual_MU), 0) AS Total_Generated_MU
FROM POWERPLANTS p
INNER JOIN ENERGYTYPE et
    ON p.Type_ID = et.Type_ID
LEFT JOIN PRODUCTIONLOG pl
    ON p.Plant_ID = pl.Plant_ID
   AND pl.Log_Date = :selected_date
WHERE p.State_Code = :state_code
GROUP BY et.Type_Name
HAVING Total_Generated_MU > 0
ORDER BY Total_Generated_MU DESC;
```

```
Get all system alerts

SELECT
    'Low Efficiency' AS Alert_Type,
    p.Plant_Name,
    s.State_Name,
    ROUND(AVG(pl.Efficiency_Percentage), 2) AS Value,
    'WARNING' AS Severity,
    MAX(pl.Log_Date) AS Last_Occurrence
FROM POWERPLANTS p
INNER JOIN PRODUCTIONLOG pl
    ON p.Plant_ID = pl.Plant_ID
INNER JOIN STATE s
    ON p.State_Code = s.State_Code
WHERE pl.Log_Date >= DATE_SUB(CURDATE(), INTERVAL 3 DAY)
GROUP BY
    p.Plant_ID,
    p.Plant_Name,
    s.State_Name
HAVING AVG(pl.Efficiency_Percentage) < 60

UNION ALL

SELECT
    'Critical Coal Stock' AS Alert_Type,
    p.Plant_Name,
    s.State_Name,
    pl.Coal_Stock_Days AS Value,
    CASE
        WHEN pl.Coal_Stock_Days < 4 THEN 'CRITICAL'
        ELSE 'WARNING'
    END AS Severity,
    pl.Log_Date AS Last_Occurrence
FROM POWERPLANTS p
INNER JOIN PRODUCTIONLOG pl
    ON p.Plant_ID = pl.Plant_ID
INNER JOIN STATE s
    ON p.State_Code = s.State_Code
INNER JOIN ENERGYTYPE et
    ON p.Type_ID = et.Type_ID
WHERE pl.Log_Date = (
        SELECT MAX(Log_Date)
        FROM PRODUCTIONLOG
        WHERE Plant_ID = p.Plant_ID
      )
  AND et.Type_Name IN ('THERMAL', 'THER (CGT)')
  AND pl.Coal_Stock_Days < 7

UNION ALL

SELECT
    'Plant Outage' AS Alert_Type,
    p.Plant_Name,
    s.State_Name,
    os.Cap_Under_Outage_MW AS Value,
    CASE
        WHEN os.Cap_Under_Outage_MW >= 500 THEN 'CRITICAL'
        ELSE 'WARNING'
    END AS Severity,
    os.Status_Date AS Last_Occurrence
FROM POWERPLANTS p
INNER JOIN OPERATIONAL_STATUS os
    ON p.Plant_ID = os.Plant_ID
INNER JOIN STATE s
    ON p.State_Code = s.State_Code
WHERE os.Status = 'Under Outage'
  AND os.Status_Date >= DATE_SUB(CURDATE(), INTERVAL 1 DAY)

ORDER BY
    Severity DESC,
    Last_Occurrence DESC;
```

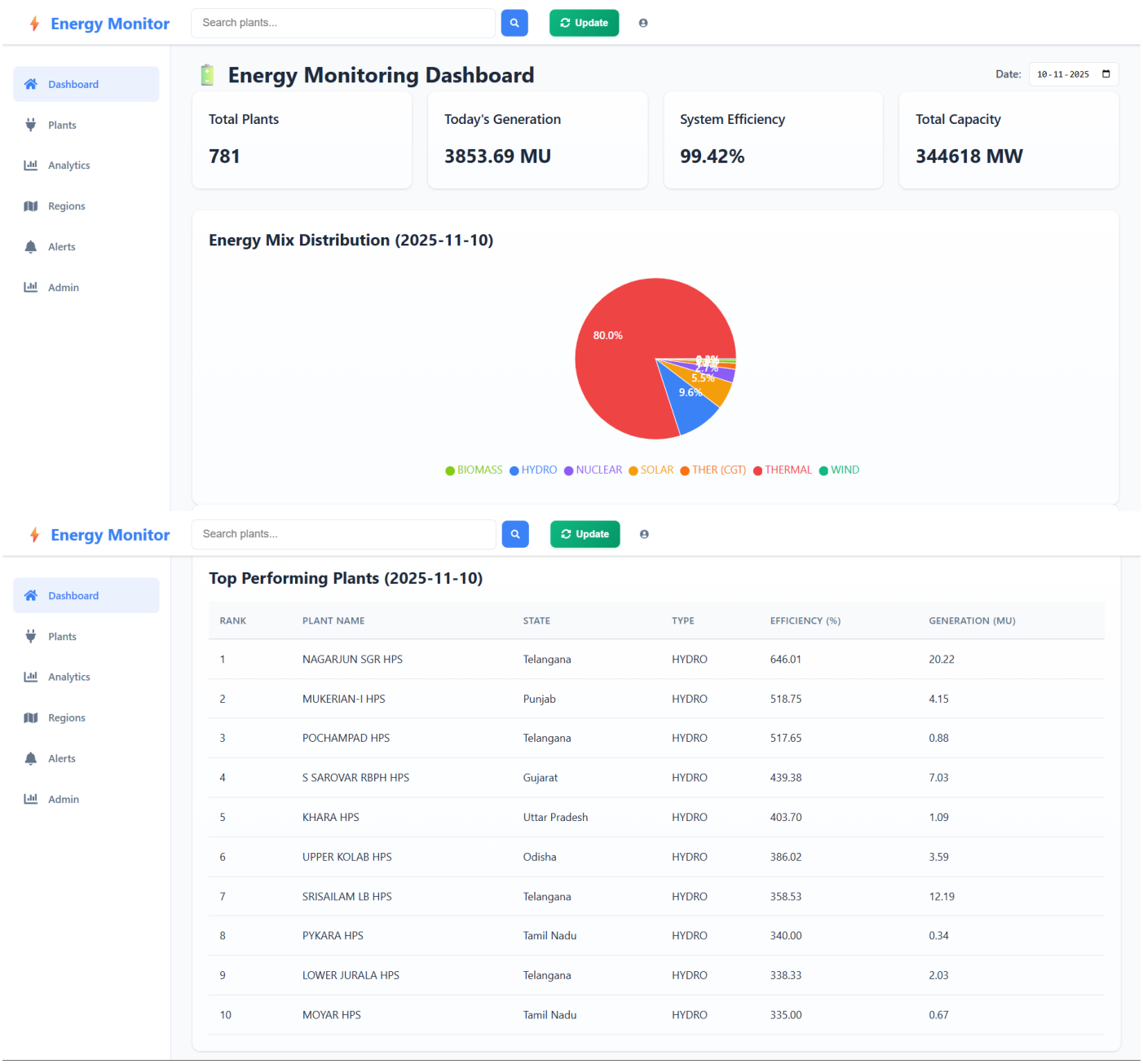
```
Get renewable vs non-renewable energy mix by state

WITH EnergyMix AS (
    SELECT
        s.State_Name,
        s.Region,
        CASE
            WHEN et.Type_Name IN ('HYDRO', 'WIND', 'SOLAR', 'BIOMASS')
                THEN 'Renewable'
            ELSE 'Non-Renewable'
        END AS Energy_Category,
        SUM(pl.Todays_Actual_MU) AS Total_Production_MU
    FROM POWERPLANTS p
    INNER JOIN PRODUCTIONLOG pl
        ON p.Plant_ID = pl.Plant_ID
    INNER JOIN STATE s
        ON p.State_Code = s.State_Code
    INNER JOIN ENERGYTYPE et
        ON p.Type_ID = et.Type_ID
    GROUP BY
        s.State_Name,
        s.Region,
        Energy_Category
)

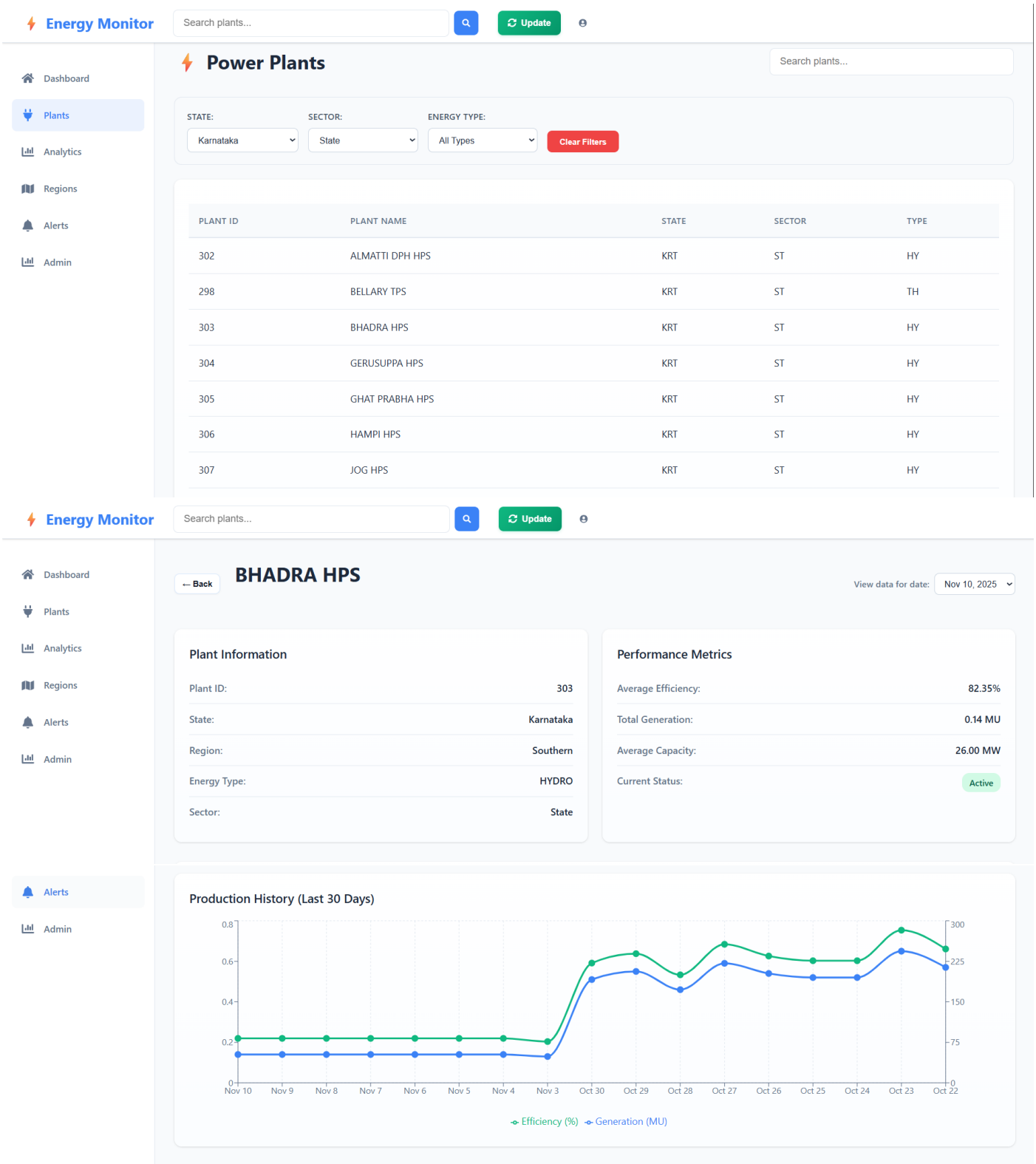
SELECT
    State_Name,
    Region,
    SUM(CASE WHEN Energy_Category = 'Renewable' THEN Total_Production_MU ELSE 0 END) AS Renewable_MU,
    SUM(CASE WHEN Energy_Category = 'Non-Renewable' THEN Total_Production_MU ELSE 0 END) AS Non_Renewable_MU,
    SUM(Total_Production_MU) AS Total_MU,
    ROUND(
        (
            SUM(CASE WHEN Energy_Category = 'Renewable' THEN Total_Production_MU ELSE 0 END)
            / NULLIF(SUM(Total_Production_MU), 0)
        ) * 100,
        2
    ) AS Renewable_Percentage
FROM EnergyMix
GROUP BY
    State_Name,
    Region
ORDER BY
    Renewable_Percentage DESC;
```

Features:

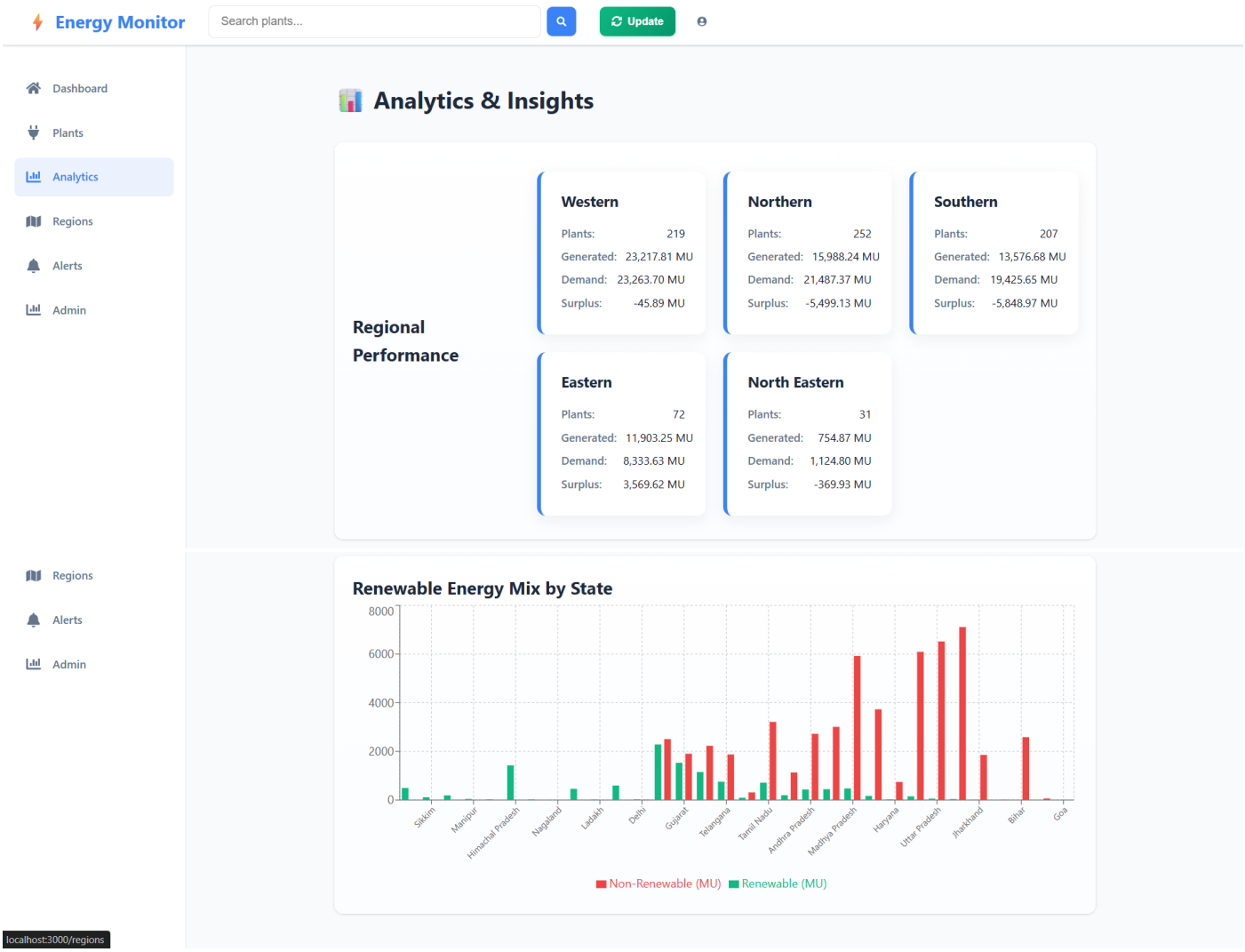
Main Dashboard:



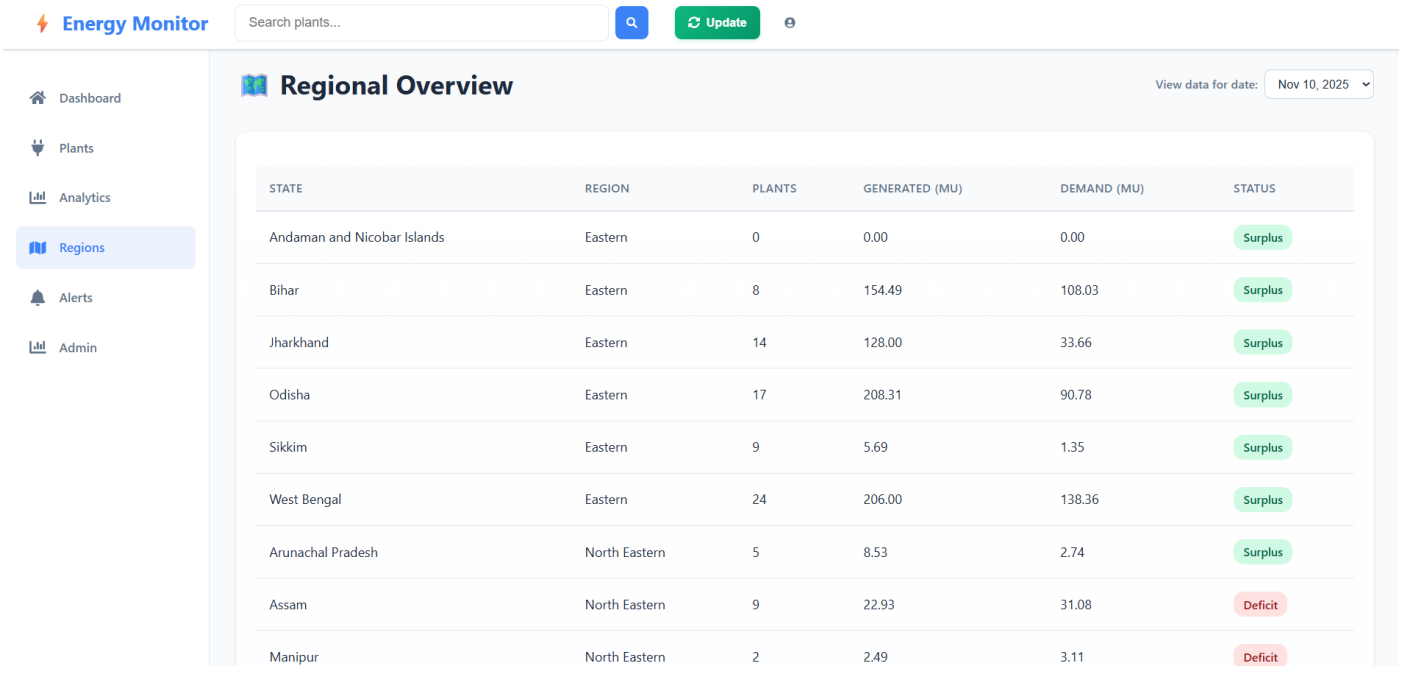
Plants Dashboard:



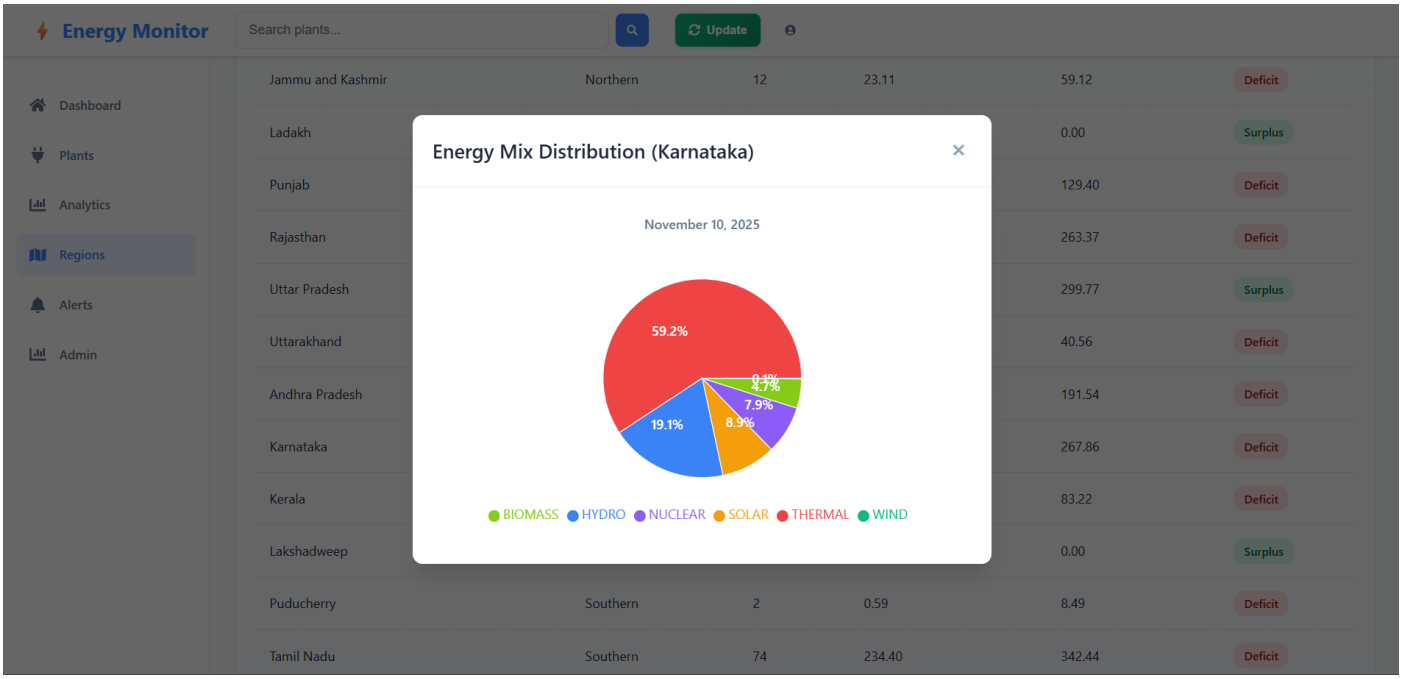
Analytics and Insights:



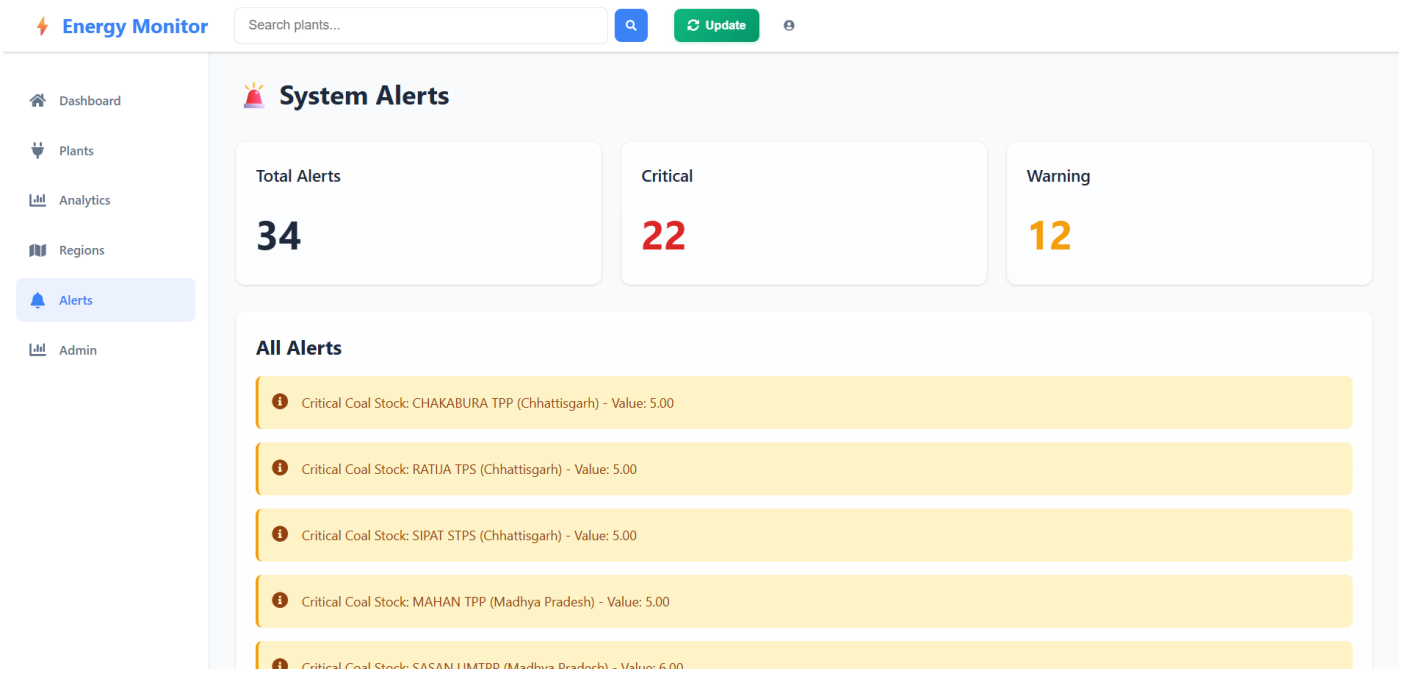
Regions Dashboard:



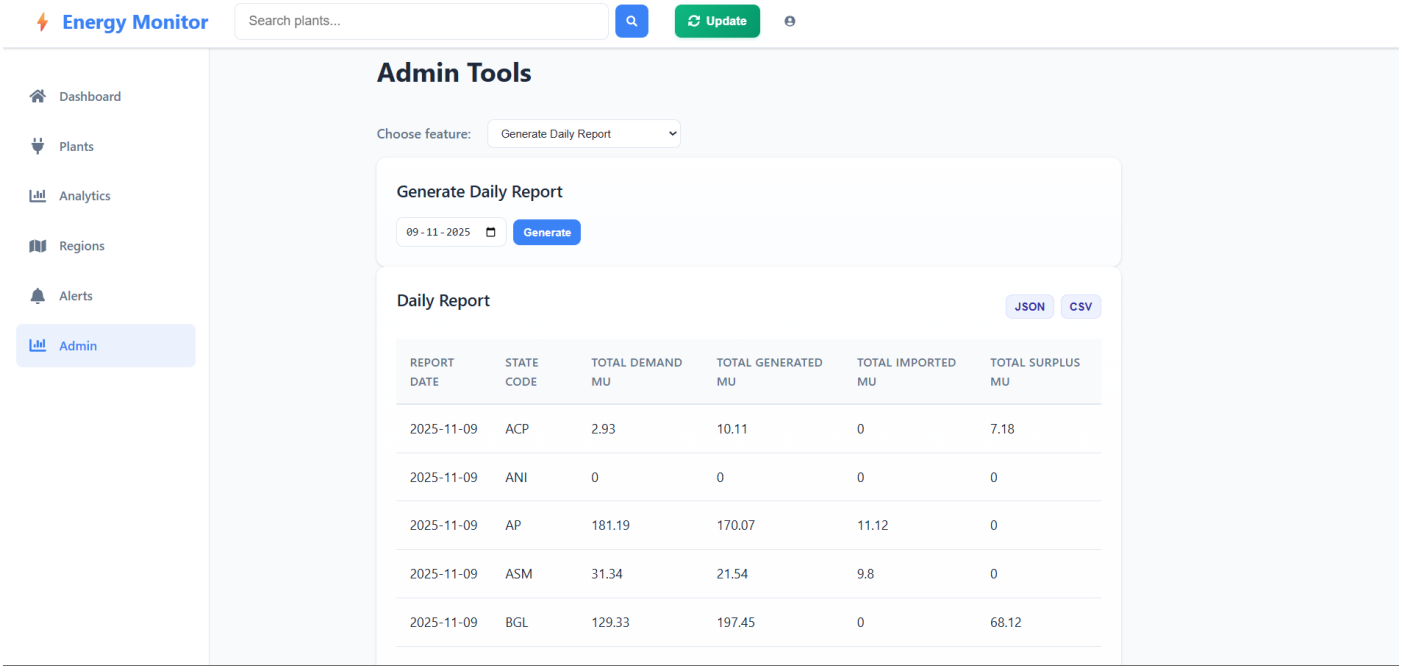
STATE	REGION	PLANTS	GENERATED (MU)	DEMAND (MU)	STATUS
Andaman and Nicobar Islands	Eastern	0	0.00	0.00	Surplus
Bihar	Eastern	8	154.49	108.03	Surplus
Jharkhand	Eastern	14	128.00	33.66	Surplus
Odisha	Eastern	17	208.31	90.78	Surplus
Sikkim	Eastern	9	5.69	1.35	Surplus
West Bengal	Eastern	24	206.00	138.36	Surplus
Arunachal Pradesh	North Eastern	5	8.53	2.74	Surplus
Assam	North Eastern	9	22.93	31.08	Deficit
Manipur	North Eastern	2	2.49	3.11	Deficit

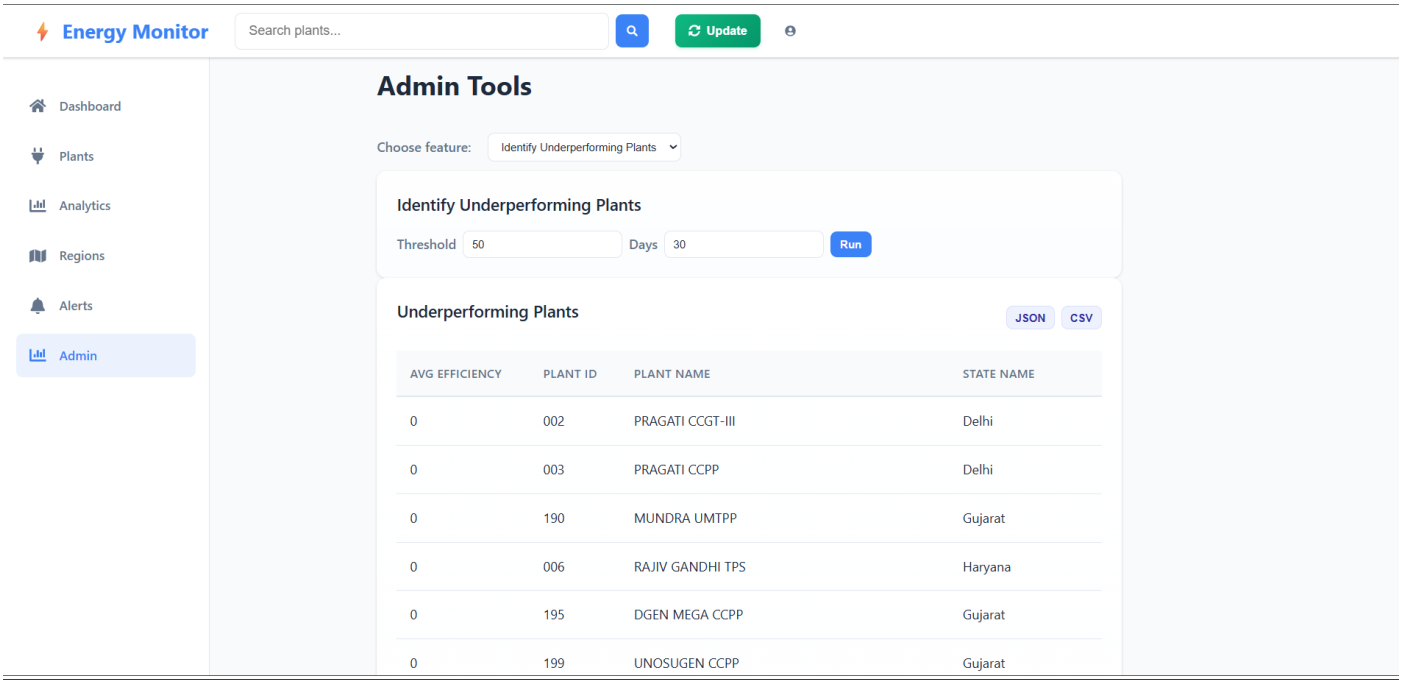


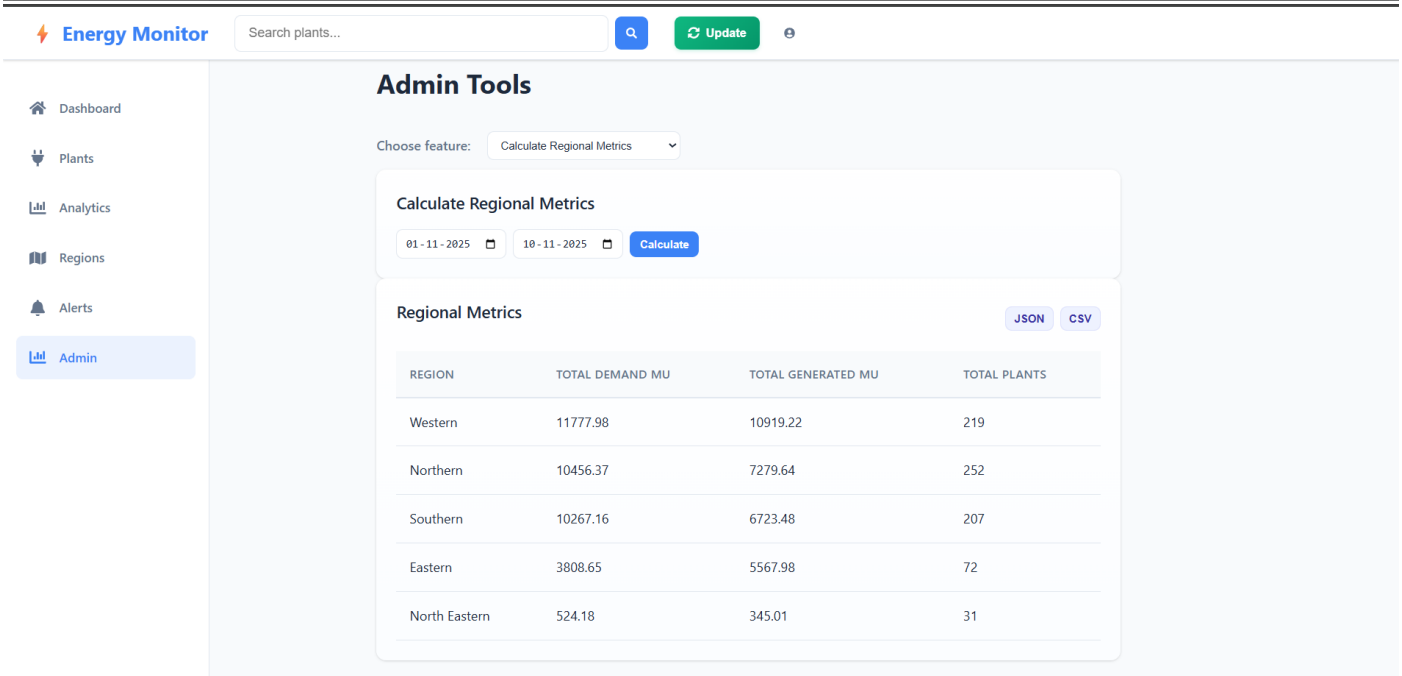
Alerts:



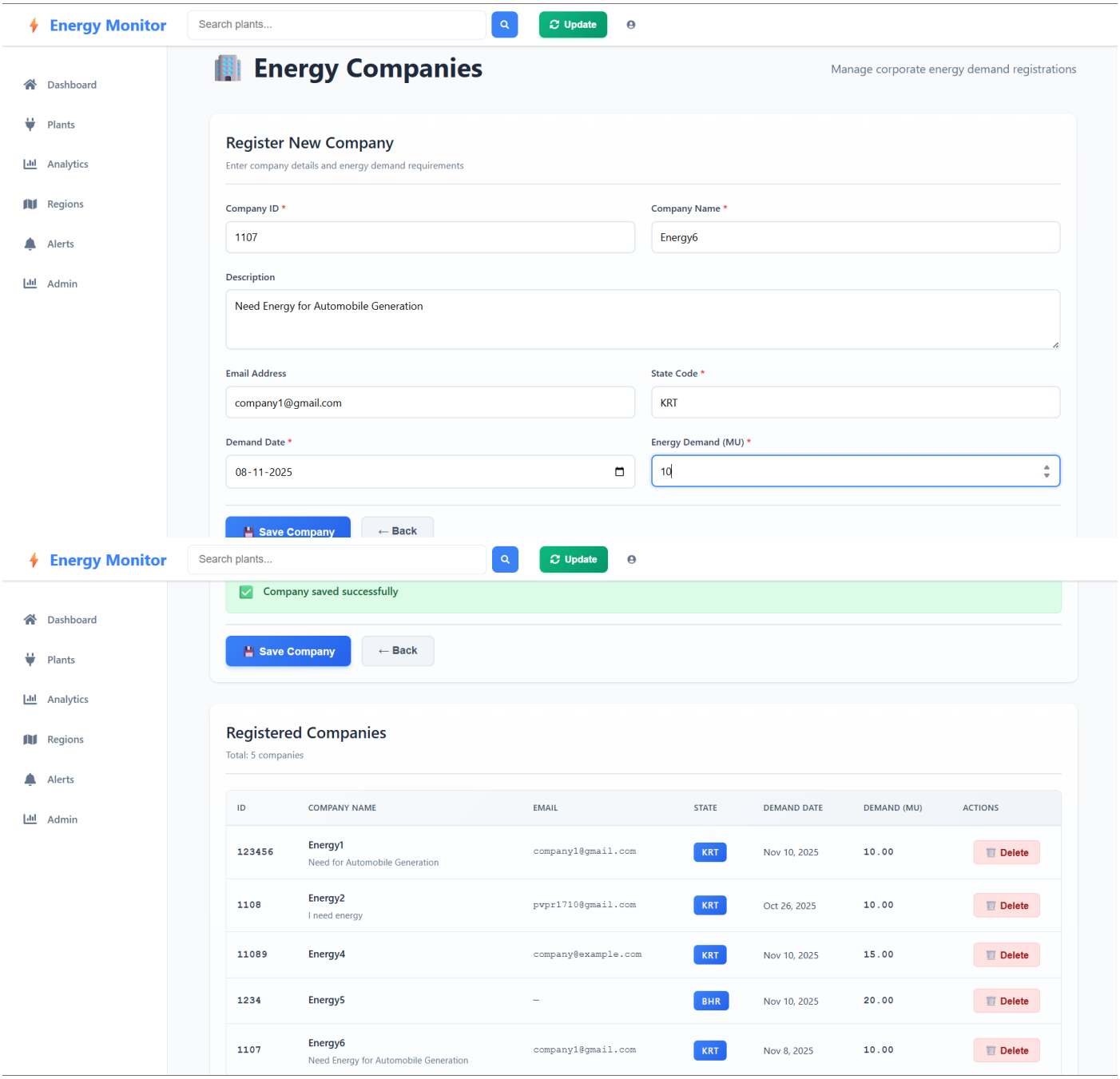
Admin Tools:


Energy Monitor
Search plants...
Update
Dashboard
Plants
Analytics
Regions
Alerts
Admin
Admin Tools
Choose feature:
Generate Daily Report
Generate Daily Report
09-11-2025
Generate
Daily Report
JSON
CSV
REPORT DATE
STATE CODE
TOTAL DEMAND MU
TOTAL GENERATED MU
TOTAL IMPORTED MU
TOTAL SURPLUS MU
2025-11-09 ACP 2.93 10.11 0 7.18
2025-11-09 ANI 0 0 0 0
2025-11-09 AP 181.19 170.07 11.12 0
2025-11-09 ASM 31.34 21.54 9.8 0
2025-11-09 BGL 129.33 197.45 0 68.12


Energy Monitor
Search plants...
Update
Dashboard
Plants
Analytics
Regions
Alerts
Admin
Admin Tools
Choose feature:
Identify Underperforming Plants
Identify Underperforming Plants
Threshold 50
Days 30
Run
Underperforming Plants
JSON
CSV
AVG EFFICIENCY
PLANT ID
PLANT NAME
STATE NAME
0 002 PRAGATI CCGT-III Delhi
0 003 PRAGATI CCPP Delhi
0 190 MUNDRA UMTPP Gujarat
0 006 RAJIV GANDHI TPS Haryana
0 195 DGEN MEGA CCPP Gujarat
0 199 UNOSUGEN CCPP Gujarat


Energy Monitor
Search plants...
Update
Dashboard
Plants
Analytics
Regions
Alerts
Admin
Admin Tools
Choose feature:
Calculate Regional Metrics
Calculate Regional Metrics
01-11-2025
10-11-2025
Calculate
Regional Metrics
JSON
CSV
REGION
TOTAL DEMAND MU
TOTAL GENERATED MU
TOTAL PLANTS
Western 11777.98 10919.22 219
Northern 10456.37 7279.64 252
Southern 10267.16 6723.48 207
Eastern 3808.65 5567.98 72
North Eastern 524.18 345.01 31

CRUD Operations:



Code Snippets:

```
proc = db.session.execute(text("CALL sp_CalculateRegionalMetrics(:start_date, :end_date)"), {'start_date': sd, 'end_date': ed})
try:
    results = proc.fetchall()
except Exception:
    cur = proc.cursor if hasattr(proc, 'cursor') else None
    if cur is not None:
        results = cur.fetchall()
    else:
        results = []
```

```
try:
    proc = db.session.execute(text("CALL sp_GenerateDailyEnergyReport(:d)"), {'d': report_date})
    try:
        rows = proc.fetchall()
    except Exception:
        cur = getattr(proc, 'cursor', None)
        rows = cur.fetchall() if cur is not None else []
```

```
procedures = [
    'sp_UpdateRegionGenerationFromProduction',
    'sp_UpdateRegionSurplusAndImports',
    'sp_CalculatePlantEfficiency',
    'sp_InsertAllMissingActiveStatuses'
]

for proc_name in procedures:
    try:
        db.session.execute(text(f"CALL {proc_name}()"))
        db.session.commit()
        sql_results.append({'procedure': proc_name, 'success': True})
        print(f"Successfully called {proc_name}")
    except Exception as e:
        db.session.rollback()
        sql_results.append({'procedure': proc_name, 'success': False, 'error': str(e)})
        print(f"Error calling {proc_name}: {e}")

results.append({
    'step': 'Database Updates',
    'success': all(r.get('success', False) for r in sql_results),
    'details': sql_results
})
```

```
try:
    threshold = float(request.args.get('threshold', 60))
    days = int(request.args.get('days', 30))

    proc = db.session.execute(text("CALL sp_IdentifyUnderperformingPlants(:threshold, :days)"), {'threshold': threshold, 'days': days})
    results = proc.fetchall()

    data = [{
        'plant_id': r[0],
        'plant_name': r[1],
        'state_name': r[2],
        'avg_efficiency': float(r[3] or 0)
    } for r in results]

    return jsonify({'success': True, 'data': data}), 200
```